BÁO CÁO THỰC HÀNH

Thực hành An Toàn Mạng Máy Tính

Lab 2: Modern Cryptography

GVTH: Nguyễn Ngọc Trưởng

Ngày báo cáo: 24/03/2026

Nhóm 11 – NT101.Q22.1

THÔNG TIN CHUNG

MSSV	Họ và tên	Nội dung báo cáo
24520435	Lê Văn Hải	Câu 1, câu 2
24520613	Nguyễn Thái Hùng	Câu 3
24520782	Lâm Mai Hồ Nhựt Khánh	Câu 5, câu 6

Task 2.1. Cấu trúc Feistel và Sự lan truyền thay đổi

Nhiệm vụ: Dựa trên Định lý 1.2, hãy hoàn thiện hàm feistel_round. Sau đó, thực hiện mã hóa hai bản rõ M1 = 0xAB và M2 = 0xAC (chỉ khác nhau 1 bit cuối). Yêu cầu: In giá trị (L, R) sau mỗi vòng của cả hai quá trình và nhận xét về sự khác biệt giữa chúng tăng dần qua từng vòng như thế nào.

Trả lời:

- Dựa vào Định lý 1.2: Cấu trúc mật mã Feistel, trong mỗi vòng i, các giá trị mới được tính như sau:

$$LE_i = RE_{i-1}$$

$$RE_i = LE_{i-1} \oplus F(RE_{i-1},\ K_i)$$

***Hình 1**. Công thức tính các giá trị mới trong mật mã Feistel.*

```python
def F(right, subkey):
    return (right ^ subkey) & 0x0F

def feistel_round(L_in, R_in, subkey):
    # todo: Triển khai LE_i = RE_{i-1} và RE_i = LE_{i-1} XOR F(...)
    L_out = R_in
    R_out = L_in ^ F(R_in, subkey)
    return L_out, R_out

def track_avalanche(msg, key):
    L, R = (msg >> 4) & 0x0F, msg & 0x0F
    subkeys = [key & 0x0F, (key >> 4) & 0x0F, (key + 1) & 0x0F, (key + 2) & 0x0F]

    print(f"Khởi tạo: L={format(L, f'0{4}b')}, R={format(R, f'0{4}b')}")
    for i in range(4):
        L, R = feistel_round(L, R, subkeys[i])
        print(f"Vòng {i+1}: L={format(L, f'0{4}b')}, R={format(R, f'0{4}b')}")
    return (L << 4) | R

# Chạy thử với 2 bản rõ khác nhau 1 bit
print("--- Mã hóa M1 ---")
track_avalanche(0xAB, 0x12)
print("--- Mã hóa M2 ---")
track_avalanche(0xAC, 0x12)
```

***Hình 2**. Đoạn code sau khi hoàn thiện hàm feistel_round.*

Chạy thử chương trình:

- Khởi tạo:
  - M1 (0xAB): Khởi tạo 1010, 1011.
  - M2 (0xAC): Khởi tạo 1010, 1100.
- Kết quả sau khi chạy chương trình:

```
PS D:\Coding\NT101\Lab02> python -u "d:\Coding\NT101\Lab02\feistel_round.py"
--- Mã hóa M1 ---
Khởi tạo: L=1010, R=1011
Vòng 1: L=1011, R=0011
Vòng 2: L=0011, R=1001
Vòng 3: L=1001, R=1001
Vòng 4: L=1001, R=0100
--- Mã hóa M2 ---
Khởi tạo: L=1010, R=1100
Vòng 1: L=1100, R=0100
Vòng 2: L=0100, R=1001
Vòng 3: L=1001, R=1110
Vòng 4: L=1110, R=0011
PS D:\Coding\NT101\Lab02>
```

***Hình 3**. Kết quả nhận được sau khi chạy chương trình.*

- Dựa trên kết quả in ra, ta có nhận xét từng vòng như sau:
  - **Vòng 1**: Sự khác biệt ban đầu giữa M1 và M2 chỉ nằm ở phần R của bản rõ. Sau vòng 1, giá trị L của hai quá trình sẽ khác nhau (vì $L_1 = R_0$). Tuy nhiên, sự khác biệt vẫn còn khu trú ở một vài bit.
  - **Vòng 2 và vòng 3**: Do hàm F sử dụng phép toán XOR và tương tác với *subkey*, sai lệch nhỏ ban đầu bắt đầu lan sang các bit khác, Mỗi khi một bit thay đổi đi qua hàm *F*, nó có khả năng làm thay đổi nhiều Bit ở vế đối diện trong vòng kế tiếp.
  - **Vòng 4**: Đến vòng cuối cùng, bạn sẽ thấy chuỗi nhị phân (L, R) của M1 và M2 khác nhau rất nhiều, mặc dù đầu vào chỉ khác nhau đúng 1-2 bit.

Task 2.2. Mode of Operation - Chế độ hoạt động

Nhiệm vụ: Sử dụng thư viện pycryptodome, hãy viết script mã hóa chuỗi văn bản có tính lặp lại cao: "UIT_LAB_UIT_LAB_UIT_LAB_UIT_LAB_". Thực hiện mã hóa bằng cả hai chế độ AES-ECB và AES-CBC (với IV cố định).

- In kết quả dưới dạng Hexadecimal và so sánh các khối 16-byte của bản mã.
- Bạn hãy tìm hiểu, thử nghiệm tất cả các Mode of Operation khác, và đưa ra nhận xét.

Trả lời:

```python
from Crypto.Cipher import AES

# Khóa 16 bytes và IV cố định (16 bytes toàn số 0)
key = b'1234567890123456'
iv = b'0' * 16
plaintext = b"UIT_LAB_UIT_LAB_UIT_LAB_UIT_LAB_" # 32 bytes (vừa khít 2 khối)

# --- 1. AES-ECB ---
cipher_ecb = AES.new(key, AES.MODE_ECB)
ct_ecb = cipher_ecb.encrypt(plaintext)

# --- 2. AES-CBC ---
cipher_cbc = AES.new(key, AES.MODE_CBC, iv)
ct_cbc = cipher_cbc.encrypt(plaintext)

# In kết quả theo khối 16-byte để so sánh
print(f"ECB: {ct_ecb.hex()}")
print(f"   Khối 1: {ct_ecb[:16].hex()}")
print(f"   Khối 2: {ct_ecb[16:].hex()}")

print(f"\nCBC: {ct_cbc.hex()}")
print(f"   Khối 1: {ct_cbc[:16].hex()}")
print(f"   Khối 2: {ct_cbc[16:].hex()}")
```

***Hình 4**. Đoạn code của chương trình mã hóa bằng AES-ECB và AES-CBC.*

Chạy thử chương trình:

```
PS D:\Coding\NT101\Lab02> python -u "d:\Coding\NT101\Lab02\ECB_CBC.py"
ECB: 02b0647de210da452e3bdcadc415814e02b0647de210da452e3bdcadc415814e
Khối 1: 02b0647de210da452e3bdcadc415814e
Khối 2: 02b0647de210da452e3bdcadc415814e

CBC: 47747530a46ebd156c1da8763c54be1b46807a7ec513e5e69d9256881a0f0c09
Khối 1: 47747530a46ebd156c1da8763c54be1b
Khối 2: 46807a7ec513e5e69d9256881a0f0c09
PS D:\Coding\NT101\Lab02>
```

***Hình 5**. Kết quả khi chạy chương trình mã hóa bằng AES-ECB và AES-CBC.*

- **Chế độ ECB**: Hai khối mã giống hệt nhau (02b0647de210da452e3bdcadc415814e). Vì bản rõ lặp lại, ECB mã hóa từng khối độc lập mà không có sự liên kết, dẫn đến bản mã cũng lặp lại.
- **Chế độ CBC**: Hai khối bản mã khác nhau hoàn toàn. Khối thứ nhất được XOR với **IV**, khối thứ hai được XOR với bản mã của khối thứ nhất trước khi mã hóa, giúp che giấu tính lặp lại.

Tìm hiểu các Mode of Operation khác:

Chế độ	Cơ chế hoạt động	Nhận xét
CTR (Counter)	Biến khối thành dòng (Stream) bằng cách mã hóa một bộ đếm (Counter).	Có thể mã hóa/giải mã song song, tốc độ cực nhanh. Không cần padding.
CFB (Cipher Feedback)	Kết quả mã hóa vòng trước làm đầu vào mã hóa vòng sau.	Biến thuật toán khối thành thuật toán dòng. Có khả năng tự đồng bộ hóa nếu mất dữ liệu trên đường truyền.
OFB (Output Feedback)	Luồng khóa được tạo ra độc lập với bản rõ.	Lỗi trên đường truyền không bị lan truyền (sai 1 bit bản mã chỉ sai 1 bit bản rõ tương ứng).
GCM (Galois/Counter Mode)	Kết hợp CTR với tính năng xác thực dữ liệu (MAC).	Vừa đảm bảo bí mật vừa đảm bảo dữ liệu không bị sửa đổi. Đây là mode được dùng trong HTTPS (TLS).

Task 2.3 Avalanche Effect

Nhiệm vụ: Viết script mã hóa hai chuỗi "STAYHOME" và "STAYHOMA" bằng thuật toán DES. Chuyển kết quả bản mã sang dạng chuỗi nhị phân và đếm số lượng bit khác nhau (Hamming Distance). Thay đổi key lần lượt với mã số sinh viên trong nhóm, tính toán tỷ lệ phần trăm thay đổi bit với các lần thay đổi và đưa ra nhận xét.

Mục tiêu bài thực hành:

- Sử dụng ngôn ngữ lập trình Python để thực hiện mã hóa dữ liệu bằng thuật toán DES
- Tìm hiểu về Khoảng cách Hamming (Hamming Distance) và cách đo lường sự thay đổi giữa hai bản mã.
- Kiểm chứng Hiệu ứng Thác đổ (Avalanche Effect): Một thay đổi nhỏ ở đầu vào dẫn đến thay đổi lớn ở đầu ra.

Trả lời:

Bước 1. Khai báo thư viện và định nghĩa hàm bổ trợ

- **from Crypto.Cipher import DES**: Khai báo thư viện mã hóa chuẩn để sử dụng thuật toán DES (Data Encryption Standard)
- **import binascii**: Đây là thư viện tiêu chuẩn của Python dùng để chuyển đổi giữa dữ liệu nhị phân (binary) và các định dạng khác như Hexadecimal (thập lục phân).
- **pad(text):** Hàm thêm khoảng trắng vào cuối chuỗi nếu chuỗi chưa đủ độ dài quy định do thuật toán yêu cầu dữ liệu đầu vào phải là bội số của 8 bytes.

```python
from Crypto.Cipher import DES
import binascii

# b ù dữ liệu cho đủ 8 bytes do DES yêu cầu khối 64 bit
def pad(text):
    while len(text) % 8 != 0:
        text += ' '
    return text.encode('utf-8')
```

Bước 2. Chuyển đổi dữ liệu sang hệ nhị phân

- **int.from_bytes(data, 'big')**: Chuyển đổi dữ liệu dạng Byte (kết quả sau mã hóa) sang một số nguyên lớn.
- **bin**(...): Chuyển số nguyên vừa tạo được sang chuỗi nhị phân (gồm các ký tự 0 và 1) để có thể đếm và so sánh từng bit. Kết quả của hàm này luôn bắt đầu bằng tiền tố '0b'.
- **[2:]**: Loại bỏ hai kí tự đầu tiên, chỉ dữ lại phần số nhị phân thực sự
- **.zfill(len(data) * 8)**: Nếu chuỗi nhị phân ngắn hơn tổng số bit yêu cầu (do các số 0 ở đầu bị lược bỏ khi chuyển sang số nguyên), hàm này sẽ tự động thêm các số 0 vào bên trái cho đủ độ dài.

```python
# chuyển dữ liệu sang chuỗi nhị phân
def get_bin(data):
    return bin(int.from_bytes(data, 'big'))[2:].zfill(len(data) * 8)
```

Bước 3. Tính toán Khoảng cách Hamming (Hamming Distance)

- **Hàm sử dụng:** sum(c1 != c2 for c1, c2 in zip(bin1, bin2))
- **Tác dụng:** So sánh vị trí tương ứng của từng bit giữa hai bản mã bin1 và bin2. Nếu bit tại cùng một vị trí khác nhau, biến đếm sẽ tăng lên 1. Kết quả cuối cùng là tổng số bit khác biệt giữa hai bản mã.

```python
# Hàm đếm số bit khác nhau giữa 2 chuỗi nhị phân
def hamming_distance(bin1, bin2):
    return sum(c1 != c2 for c1, c2 in zip(bin1, bin2))
```

Bước 4: Thực hiện mã hóa với các key (MSSV)

- **Hàm sử dụng:** DES.new(key, DES.MODE_ECB) và cipher.encrypt()
- **Tác dụng:** Khởi tạo bộ mã hóa DES với khóa (Key) là mã số sinh viên và thực hiện mã hóa hai chuỗi mục tiêu "STAYHOME" và "STAYHOMA".

```python
mssv_list = ["24520613", "24520435 "]
str1 = "STAYHOME"
str2 = "STAYHOMA"

print(f"{'MSSV (Key)':<15} | {'Hamming Distance':<18} | {'Tỷ lệ thay đổi':<15}")

for mssv in mssv_list:
    # Lấy 8 ký tự đầu của MSSV làm Key
    key = mssv.encode('utf-8')[:8]
    cipher = DES.new(key, DES.MODE_ECB)

    # Thực hiện mã hóa
    ct1 = cipher.encrypt(pad(str1))
    ct2 = cipher.encrypt(pad(str2))

    # Chuyển kết quả sang nhị phân
    bin1 = get_bin(ct1)
    bin2 = get_bin(ct2)

    # Tính khoảng cách Hamming
    dist = hamming_distance(bin1, bin2)
    ratio = (dist / 64) * 100

    print(f"{mssv:<15} | {dist:<18} | {ratio:.2f}%")
```

Kết quả sau khi chạy chương trình

```
MSSV (Key)      | Hamming Distance   | Tỷ lệ thay đổi
24520613        | 41                 | 64.06%
24520435        | 36                 | 56.25%
```

Nhận xét: Mặc dù đầu vào gần như giống hệt nhau, nhưng bản mã đầu ra lại khác biệt rất lớn (lần lượt là 41 bit và 36 bit trên tổng số 64 bit). Kết quả thực nghiệm cho thấy tỷ lệ thay đổi bit luôn dao động quanh mức 50% - 65%. Điều này minh chứng cho Hiệu ứng Thác đổ (Avalanche Effect) của thuật toán DES.

Kết luận: Thuật toán DES đảm bảo tính an toàn vì nó làm xáo trộn dữ liệu đến mức kẻ tấn công không thể dựa vào sự tương đồng của các bản rõ để suy luận ra quy luật của bản mã hay khóa mã hóa.

Task 2.5

1. So sánh độ mạnh và sự khác biệt của DES, 3DES và AES:

Đặc điểm	**DES (Data Encryption Standard)**	**3DES (Triple DES)**	**AES (Advanced Encryption Standard)**
Năm ra đời	1977	1998	2001 (Chuẩn hiện hành)
Kích thước khóa	56 bits	112 bits hoặc 168 bits	128, 192, hoặc 256 bits
Kích thước khối	64 bits	64 bits	128 bits

Cấu trúc	Mạng Feistel (16 vòng)	Mạng Feistel (48 vòng)	Mạng SPN (Substitution-Permutation)
Bảo mật	**Yếu.** Dễ dàng bị phá vỡ bằng tấn công vét cạn (Brute-force) trên phần cứng hiện đại.	**Trung bình.** An toàn hơn DES nhưng kích thước khối nhỏ (64-bit) dễ bị tấn công va chạm (Sweet32 attack).	**Rất mạnh.** Chưa có phương pháp thực tế nào phá được AES. Dùng chuẩn trong chính phủ và quân sự.
Hiệu năng	Nhanh nhưng lỗi thời.	Rất chậm (mã hóa 3 lần cho mỗi khối).	Rất nhanh, tối ưu hóa tốt trên cả phần mềm và phần cứng.

Không nên dùng Double DES vì nó chỉ có độ an toàn tương đương 2^{57} thay vì 2^{112} như kỳ vọng. Nguyên nhân là do cuộc tấn công "gặp nhau ở giữa" (meet-in-the-middle): kẻ tấn công có thể mã hóa một bản rõ với mọi khóa thứ nhất, đồng thời giải mã bản mã với mọi khóa thứ hai, rồi tìm điểm trùng khớp để suy ra cặp khóa. 3DES sử dụng ba lần mã hóa (thường là mã hóa-giải mã-mã hóa) giúp ngăn chặn kiểu tấn công này, đạt độ an toàn thực tế 2^{112}, đủ mạnh cho nhiều ứng dụng.

Vì thế nên sử dụng n-DES ($n \geq 3$) vì một DES đơn giản quá yếu (khóa 56 bit có thể bị kẻ tấn công quyết tâm ép buộc), còn Double DES thì không hông mang lại mức độ tăng cường bảo mật

Task 2.6

1. Hàm tính lũy thừa Modulo: power_mod(base, exponent, modulus)

Mục đích: Tính toán biểu thức base^exponent(mod modulus) (ví dụ 7^40 mod19) một cách hiệu quả và nhanh chóng cho các số nguyên cực lớn (như khóa RSA 1024-bit).

Thuật toán sử dụng: Square-and-Multiply (Bình phương và Nhân).

Giải thích chi tiết:

- Trong mật mã học RSA, số mũ thường rất lớn. Nếu tính base^exponent(mod modulus) trực tiếp thì giá trị sinh ra sẽ khổng lồ, gây tràn bộ nhớ (overflow) và tốn vô số thời gian tính toán.
- Hàm này giải quyết vấn đề bằng cách duyệt qua từng bit của số mũ (từ phải sang trái) thông qua phép dịch bit exponent >> 1.
- Ở mỗi bước, cơ số được bình phương lên và lấy modulo ngay lập tức (base * base) % modulus. Nhờ việc lấy modulo liên tục, các con số luôn được giữ ở mức nhỏ (nhỏ hơn modulus).
- Nếu bit hiện tại của số mũ là 1 (kiểm tra bằng exponent % 2 == 1), kết quả sẽ được nhân với cơ số hiện tại và cũng lấy modulo (result * base) % modulus..

```python
task2.6.py > power_mod
import random

def power_mod(base, exponent, modulus):

    if modulus == 1:
        return 0

    result = 1
    base = base % modulus

    while exponent > 0:
        if (exponent % 2) == 1:
            result = (result * base) % modulus
        exponent = exponent >> 1
        base = (base * base) % modulus

    return result
```

2. Hàm kiểm tra tính nguyên tố: is_prime_miller_rabin(n, k=40)

Mục đích: Kiểm tra xem một số nguyên dương n có phải là số nguyên tố hay không.

Thuật toán sử dụng: Thuật toán kiểm tra nguyên tố xác suất Miller-Rabin.

Giải thích chi tiết:

- Kiểm tra chia thử nghiệm nghiệm ngây thơ (trial division) không thể áp dụng cho các số có kích thước 64 bit hay 1024 bit vì độ phức tạp thời gian quá lớn.
- Thuật toán Miller-Rabin phân tích số chẵn n - 1 thành dạng 2^r x d (với d là số lẻ). Điều này được thực hiện trong vòng lặp while d % 2 == 0.
- Tiếp theo, thuật toán chọn một cơ số a ngẫu nhiên trong khoảng [2, n-2] và tính x = a^d mod n.
- Thuật toán kiểm tra điều kiện của định lý: nếu x ≡ 1 hoặc x ≡ -1 mod n, nó tiếp tục bình phương x lên r-1 lần. Nếu sau các phép thử mà n vẫn không biểu hiện tính chất của hợp số, nó được coi là số nguyên tố.

```python

def is_prime_miller_rabin(n, k=40):
    if n == 2 or n == 3:
        return True
    if n <= 1 or n % 2 == 0:
        return False
    r = 0
    d = n - 1
    while d % 2 == 0:
        r += 1
        d //= 2

    for _ in range(k):
        a = random.randint(2, n - 2)
        x = power_mod(a, d, n)

        if x == 1 or x == n - 1:
            continue

        is_composite = True
        for _ in range(r - 1):
            x = power_mod(x, 2, n)
            if x == n - 1:
                is_composite = False
                break

        if is_composite:
            return False

    return True
```

3. Hàm tạo số nguyên tố ngẫu nhiên: generate_random_prime(bits)

Mục đích: Sinh ra một số nguyên tố ngẫu nhiên với độ dài bit được chỉ định trước (8, 16, 64 hoặc 1024 bit).

Giải thích chi tiết:

- Đầu tiên, nó tạo một số nguyên ngẫu nhiên có độ dài bits thông qua random.getrandbits(bits).
- Phép toán bitwise p |= (1 << (bits - 1))|1 thực hiện hai nhiệm vụ thiết yếu:

  + |1: Ép bit trong cùng bên phải (Least Significant Bit - LSB) thành 1, đảm bảo số sinh ra luôn là số lẻ (số nguyên tố lớn hơn 2 luôn là số lẻ).

+ |(1 << (bits - 1)): Ép bit ngoài cùng bên trái (Most Significant Bit - MSB) thành 1, đảm bảo số sinh ra có độ dài chuẩn xác bằng đúng biến bits được yêu cầu (không bị ngắn hơn do các số 0 vô nghĩa ở đầu).

- Cuối cùng, đẩy số vừa tạo vào hàm is_prime_miller_rabin. Nếu trả về True, vòng lặp dừng và trả về số nguyên tố đó

```
def generate_random_prime(bits):
    while True:
        p = random.getrandbits(bits)
        p |= (1 << (bits - 1)) | 1

        if is_prime_miller_rabin(p):
            return p
```

4. Hàm tìm ước chung lớn nhất: gcd(a, b)

Mục đích: Tìm ước chung lớn nhất của hai số nguyên a và b.

Thuật toán sử dụng: Thuật toán Euclid.

Giải thích chi tiết:

- Hàm dựa trên nguyên lý cơ bản của Euclid: GCD(a, b) = GCD(b, a mod b).
- Thay vì trừ lặp lại, thuật toán dùng phép chia lấy dư (%) liên tục gán a = b và b = a mod b cho đến khi b giảm về 0.
- Lúc này, phần dư cuối cùng khác không (tức là giá trị a hiện tại) chính là ước chung lớn nhất.

```
def gcd(a, b):
    while b != 0:
        a, b = b, a % b
    return a
```

Kiểm thử chương trình:

1. Số nguyên tố

- Tạo ra một số nguyên tố ngẫu nhiên có: 8 bits ,16 bits ,64 bits
- Xác định **10 số nguyên tố lớn nhất nhỏ hơn số nguyên tố Mersenne thứ 10**.
- Kiểm tra xem một số nguyên tùy ý nhỏ hơn 2^89- 1 có phải là số nguyên tố hay không.

```
--- YÊU CẦU 1: SỐ NGUYÊN TỐ ---
Số nguyên tố 8 bits: 199
Số nguyên tố 16 bits: 46769
Số nguyên tố 64 bits: 12758171784263877131

Số nguyên tố Mersenne thứ 10 (M_89) = 618970019642690137449562111
10 số nguyên tố lớn nhất nhỏ hơn M_89:
1. 618970019642690137449562091
2. 618970019642690137449562081
3. 618970019642690137449562063
4. 618970019642690137449562043
5. 618970019642690137449562013
6. 618970019642690137449562009
7. 618970019642690137449561847
8. 618970019642690137449561791
9. 618970019642690137449561671
10. 618970019642690137449561509

Kiểm tra số ngẫu nhiên < 2^89 - 1:
Số: 439889929080803192136534870
Là số nguyên tố? False
```

2.Xác định ước số chung lớn nhất (GCD) của hai số nguyên lớn tùy ý (có kích thước lớn nhất mà chương trình có thể xử lý, ví dụ 1024-bit).

```
--- YÊU CẦU 2: ƯỚC CHUNG LỚN NHẤT (GCD) ---
Số A (1024 bits): 172575299343159793495786426012849162725096677418387441998230830942025536049510443991
908598263720190578769724078496642737838161786413712865801835171564615921937205650539035448282514599355
703477344710301931857768748741107695988999362973543899741203839230522195509417171809344622728128374060
23807925620583301658 5
Số B (1024 bits): 158164409138324485078505204959984703961209517156675766018238554349350917497001344439
524326411020177082679088507673944237715623295205586556503141461469586657902635891085853560724724737 6
038871484867362353250522886770565937215660770270464308064286759491709965948724824377078529900921999 38
646905614552207479714755
GCD(A, B) = 5
```

3.Tính lũy thừa theo modulo:

- Tính toán giá trị: a^x mod p

- Chương trình phải có khả năng tính toán với: Số mũ lớn (x>40)

Ví dụ kiểm thử: 7^40 mod 19

```
--- YÊU CẦU 3: TÍNH LŨY THỪA MODULO ---
Ví dụ: 7^40 mod 19 = 7

Tính toán số lớn (1024 bits):
Kết quả = 559490831971003611810814812465940778777334467186006992246945743232094726803110770486639335 0
926801263409097303972997129619004977361049831268147491762769723067088030295303060576857485849612971 09
8919787258679024317440575425742223548194663022370191863365854827190590628148736035717824924989333566 8
016429022884005
PS D:\codeblock>
```

HẾT